Local Models, Global Risk: Assessing Emerging Threats in Local AI APIs in Browsers

Zahra Moti[1], Mina Mehrvarz[1], Tim Vlummens[2], Gautham Shaji[1], and Gunes Acar[1]

[1] Radboud University
[2] COSIC, KU Leuven

Abstract. Recently introduced built-in AI APIs in browsers allow websites and extensions to use on-device large language models (LLMs) for tasks such as summarization, translation, and general-purpose prompting. While these APIs offer a local and privacy-preserving alternative to server-side inference, exposing these APIs to untrusted web content brings novel risks that need to be studied. In this paper, we investigate the security and privacy implications of built-in AI APIs shipped in Chrome and Edge browsers through controlled experiments and web measurements. In controlled susceptibility tests of the Summarizer and Prompt APIs, we show that local model's outputs can be manipulated through different forms of prompt injection, including hidden text that is invisible to users. We then demonstrate how adversarial user-generated content, such as reviews and comments can distort summaries through narrative manipulation. We quantify the susceptibility of two browsers to these attacks under different scenarios. Turning to privacy risks, we show how malicious scripts can use the Prompt API to locally infer sensitive user attributes such as pregnancy status or sensitive health issues. Complementing these experiments, we measure AI API usage on the top 50K websites, finding limited but diverse use cases including review summarization, ad-related keyword extraction, and shopping assistants. Finally, motivated by the lack of transparency around the AI API usage, we introduce AI Guardian, a browser extension that intercepts and reveals built-in AI API calls by websites, while detecting prompt-injection attempts. Our research points to a need for transparency and safeguards before browser-integrated AI APIs see broad adoption across the web.

1 Introduction

Recent advances in large language models have made it possible to run complex models directly on end-user devices. In 2024, Google introduced Chrome Built-in AI [10], a set of browser APIs that exposes on-device LLMs (e.g., Gemini Nano) to web pages and browser extensions. Similarly, Microsoft has begun integrating on-device AI capabilities into the Edge browser, using the same experimental APIs as Chrome [2]. Through local language models, these APIs provide a privacy-friendly alternative that reduces the dependency on online AI platforms.

Embedding language models directly into the browser introduces new security and privacy challenges. While inference is performed locally, exposing AI functionality to web pages and extensions opens a new attack surface. Untrusted web scripts can invoke on-device models on demand, potentially operating over sensitive web data. Additional risks include stealthy model invocations, manipulation of model outputs to influence user behavior, and using model capabilities for browser fingerprinting purposes.

These risks are compounded by a lack of transparency around API usage. Unlike traditional browser capabilities such as camera, microphone, or geolocation access, on-device AI APIs do not require user permission and they lack UI indicators. While the first interaction with an on-device AI API involves model initialization and download triggered by a user activation such as a click, subsequent invocations can occur programmatically and silently in the background. Both Chrome and Edge provide developer-oriented diagnostic logs for on-device AI activity, but these lack critical information such as which website or script called the APIs. As a result, potentially sensitive AI interactions may occur without user awareness or oversight.

Locally deployed models introduce another weakness: they lack centralized abuse monitoring and oversight typically present in server-side systems. Although on-device models operate locally, their outputs can directly shape user-facing content, creating opportunities for subtle influence over AI-assisted interactions. Web pages may embed instructions or adversarial inputs that steer model behavior in ways users do not anticipate, including both explicit prompt-injection patterns and more subtle narrative manipulation through user-generated content such as reviews and comments. While prior research has extensively studied prompt injection in agentic systems [7, 11, 24], the browser-integrated model introduces a different attack surface. In this model, language-model functionality is exposed as a native web capability to untrusted scripts and content from arbitrary websites and extensions. This integration embeds AI directly into the web platform, enabling novel privacy, security and autonomy attacks, or amplifying existing threats. Moreover, it remains unclear whether the security and robustness of the local models are consistent across browsers.

To address these gaps, we investigate risks emanating from on-device AI APIs, focusing on Chrome and Microsoft Edge's implementations. These AI integrations have widespread privacy and security implications as the two browsers make up more than 82% of the desktop browser market share [3]. At the same time, these APIs are in an early deployment phase, and their adoption and susceptibility to different misuses remain largely unexplored. In this paper, we investigate built-in AI APIs from three perspectives: their susceptibility to manipulation by untrusted web content, their potential misuse for inferring sensitive attributes, and their real-world deployment through large-scale web crawls. To address transparency, privacy, and security concerns we introduce a browser extension that monitors on-device AI API usage, alerts users, and flags potentially malicious prompts. Our key contributions are as follows:

- We demonstrate, through controlled experiments involving Chrome and Edge, that built-in language models are susceptible to manipulation by untrusted web content. Specifically, we evaluate the models against prompt-injection attacks, involving hidden instructions and user-generated content. We show how model outputs such as summaries can be hijacked or biased.
- We show that the Prompt API can be used to infer sensitive attributes such as pregnancy status and assign medical diagnosis codes based on webpage content. This enables a novel form of local behavioral profiling that can be used to selectively exfiltrate personal data to evade detection. Moreover, we show that built-in local models can be misused to extract highly specific personal information and identifiers from webpage content.
- We provide the first large-scale empirical analysis of local AI API adoption on the web, quantifying real-world usage on 50,000 of the most popular Google CrUX domains [14].
- We develop AI Guardian, a browser extension that provides real-time visibility into local AI API invocations, and intercepts adversarial prompts through a combination of classification methods.

2 Background

This section provides background on large language models in web environments, AI-powered browser extensions, and the emergence of on-device inference.

2.1 Large Language Models

Large Language Models (LLMs) have been integrated into many modern software systems to support tasks such as text generation, summarization, translation, and conversational interaction. Traditionally, LLM capabilities have been delivered through cloud-based services, where applications send user inputs to remote servers for processing and receive generated responses. While this model provides access to powerful models running on hyperscale GPU clusters, it also introduces privacy concerns, as potentially sensitive user data must be transmitted to the server-side. Additionally, connectivity requirements and reliance on third-party providers may compromise overall reliability and data sovereignty. To address the privacy and performance limitations of cloud-based inference, there has been a growing shift toward executing models locally on user devices. Client-side AI reduces reliance on remote infrastructure by performing inference directly on local hardware. This trend is reflected in popular open-source projects such as Ollama and llama.cpp, which allow users to locally run language models and use them in other programs through Python or similar language bindings [16, 26].

2.2 AI in Browser Extensions

Web browsers serve as the primary interface between users and online content, making them a natural platform for integrating AI capabilities. In recent years,

Table 1: Availability of built-in AI APIs across Chrome and Edge. Edge APIs are available as developer previews in Canary/Dev channels. *: Available to extensions; Origin-trialled for websites.

API	Chrome	Edge
Translator	Chrome 138	Edge 143+
Language Detector	Chrome 138	Edge 147+
Summarizer	Chrome 138	Edge 138+
Writer	Developer Trial	Edge 138+
Rewriter	Developer Trial	Edge 138+
Prompt	Chrome 138*	Edge 138+
Proofreader	Origin Trial	Edge 142+

browser extensions have increasingly integrated AI-driven features such as webpage summarization, writing assistance, and translation [31]. These assistants often operate with elevated privileges and broad access to browsing activity, enabling them to observe visited pages, search queries, and form inputs. While this access enables personalized assistants, it also introduces significant privacy and security concerns. Many existing AI extensions rely on server-side processing, where webpage content and user inputs are transmitted to external providers for responses. Prior work shows that such extensions may collect extensive browsing data and contextual information to support personalization, increasing the risks of data exposure, profiling, and unintended information disclosure [31].

2.3 Built-in AI APIs in Browsers

Recent browser developments integrate LLM capabilities directly into the client. In 2024, Google introduced built-in AI APIs in Google Chrome, exposing on-device models (e.g., Gemini Nano) to web pages and extensions [10]. In parallel, Microsoft introduced similar capabilities in Microsoft Edge, providing developer-preview APIs in Canary and Dev channels for interacting with an on-device small language model (Phi-4-mini) via JavaScript [1].

These APIs enable web applications to interact with local models without transmitting page content or prompts to external servers. In this architecture, a webpage or extension invokes a browser-provided API, which interfaces with a locally stored model and returns the generated result. For security reasons, these APIs do not have direct access to page content or the DOM; instead, calling scripts must extract and provide the text to be processed (e.g., summarized or translated). After an initial model download, subsequent inference occurs entirely on-device. In Chrome, these APIs were initially introduced behind experimental flags and have gradually expanded, with several becoming available in stable releases (e.g., Chrome 138) [10]. Microsoft Edge exposes comparable functionality and APIs, currently available as developer previews [1]. Table 1 shows the availability of built-in AI APIs across Chrome and Edge.

Using the APIs, web applications, and browser extensions may check model availability, create a session, and invoke task-specific API methods to perform inference. The Translator API performs on-device language translation, whereas the LanguageDetector API determines the language of input text without generating new content. The Prompt API provides general-purpose text generation capabilities, while the Summarizer API condenses the given text into shorter representations. Although the APIs themselves cannot directly access page content, scripts executing in the page context can collect both visible and hidden text before submitting it for inference. As a result, injected or hidden instructions embedded in webpage content may influence model inputs and outputs once passed to the API. Among the available interfaces, the Prompt API and Summarizer API are particularly relevant from a security and privacy perspective due to their ability to generate outputs that may influence user perception and decisions. This study, therefore, mostly focuses on these two APIs to examine their deployment, usage, and associated privacy, security, and transparency implications.

3 Related Work

A growing body of research has demonstrated that large language models introduce new security risks when integrated into real-world applications. Studies on prompt injection and jailbreak attacks showed that model behavior can be manipulated through crafted inputs, enabling system prompt extraction, instruction hijacking, and unintended task execution [19, 33, 38].

Subsequent work examined LLM-powered agents that interacted with external tools, browsed the web, and processed untrusted content. These systems were shown to be vulnerable to indirect prompt injection and privacy leakage when exposed to adversarial data sources [6,7,24]. Proposed mitigations included input/output firewalls, capability-based designs, and sandboxed execution models [9, 11, 35]. Additional studies showed that agents could leak sensitive information even in benign settings or during reasoning, highlighting the difficulty of enforcing data minimization and privacy-aware behavior [32, 37]. While this line of work primarily focused on agentic systems that autonomously retrieved web content, built-in browser AI allows web pages to directly invoke on-device models through browser APIs, creating a distinct attack surface.

Recent work also examined the privacy implications of AI-powered assistants embedded in browser extensions. Generative AI assistants were shown to collect extensive browsing data to support personalization, raising concerns about tracking, profiling, and unintended information disclosure [31]. Prior studies further showed that browser extensions can access sensitive information from webpage content, user inputs and browser-provided data sources, enabling large-scale data collection and potential leakage to third-party services [15, 34]. A parallel line of research showed how various browser APIs, hardware features, and operating system settings could be used for device and browser fingerprinting [20–22, 29]. Due to their relatively recent introduction, the risks arising from built-in browser

AI APIs remain unexplored. Unlike prior settings, built-in browser AI APIs expose local language models to untrusted web content without user permissions. This work investigates security, privacy, and transparency risks emanating from this change.

4 Methods

To investigate the security and privacy implications of built-in AI APIs, we adopt a three-pronged methodology. First, we conduct controlled susceptibility testing to examine how untrusted input influences on-device model outputs. Second, we perform large-scale web measurements to quantify and characterize real-world adoption of the APIs. Finally, we design and implement AI Guardian, a browser extension that monitors and exposes API invocations, while detecting and blocking malicious prompts.

4.1 Susceptibility Testing

We first evaluate whether built-in AI APIs are susceptible to manipulation or misuse when operating over untrusted web content. Our analysis focuses on the Summarizer and Prompt APIs, as they expose instruction-following, open-ended inference over potentially untrusted text and are most directly susceptible to content-based manipulation, including prompt injection and narrative biasing.

Adversary model. We consider an adversary capable of injecting or influencing content within a webpage viewed by users. For example, the adversary may hide malicious content in the DOM if they control the website. They may also embed their malicious input in a comment, product review, or an article they post on a website under someone else's control. The adversary may also use third-party advertising scripts to launch attack on different websites.

Test scenarios. To evaluate susceptibility to different attacks, we built controlled HTML test pages representing three attack types:

(1) *Indirect prompt injection.* Pages embedding adversarial instructions within document content, including hidden text and structured injection patterns, to evaluate whether the model follows these instructions and overrides the intended behavior.

(2) *Narrative manipulation.* Pages simulating user-generated content (e.g., review lists) with a single adversarial entry, varying its position and proportion within the content to assess whether summaries are disproportionately influenced or dominated by the injected narrative.

(3) *Sensitive inferences.* Pages containing sensitive information or identifiers, to evaluate whether the AI APIs can be misused for selective and stealthy profiling and data exfiltration.

Evaluation procedure For each scenario, we evaluated both Chrome and Microsoft Edge implementations of the built-in AI APIs under controlled conditions. Experiments on Chrome were conducted on a MacBook with M1 processor, using Chrome version 147.0.7727.101 and the default language model it ships:

Gemini Nano (v3Nano), version 2025.06.30.1229. Experiments on Edge were conducted on Windows 11 using Edge 147.0.3912.60, which uses the Phi-4-mini-instruct version 2025.10.23.1 as the default model. To ensure reproducibility, we developed lightweight automation tools (browser scripts and extensions) to execute experiments over predefined test pages and collect outputs systematically. For each scenario, we invoked the Summarizer or Prompt API under fixed settings and recorded the outputs. For the Summarizer API, we used `type=headline`, `length=short`, and `format=markdown`; for the Prompt API, we used `temperature=0` and `top_k=3`. Each configuration was executed multiple times to account for model non-determinism, and results were evaluated using task-specific quantitative metrics (e.g., attack success rate and extraction accuracy).

Indirect prompt injection. We evaluate 12 injection variants spanning two attack families established in prior work. (i) Context-ignoring: explicit commands to disregard preceding context (e.g., "ignore all previous instructions"), corresponding to *Context Ignoring* from Liu et al. [23] and *Ignore* attack from StruQ [8]; we vary placement (prefix vs. suffix), injection-to-content ratio, and repetition to stress position- and frequency-based effects. (ii) Structure-based: fabricated document boundaries (`-- END OF DOCUMENT --`), fenced pseudo-system blocks (`""" SYSTEM PROMPT """`, `[System: ...]`), input/output pseudo-templates, and fake instruction delimiters (`### Instruction:`, `Summary: ...New Task:`), which instantiate Liu et al.'s *Separator Component* and StruQ's *Completion-Close* and *Completion-Other* attacks. Each page embeds a marker phrase (e.g., *you got hacked*); a trial succeeds if the summarizer follows the malicious instructions and the marker appears in the generated headline. Five trials per page yield 60 Chrome headlines and 60 Edge headlines.

Narrative manipulation. We evaluated the susceptibility of built-in AI APIs to narrative manipulation by measuring the attack success rate (ASR) across a suite of 12 hotel-review pages. The experimental design varied the review-set size $N \in \{5, 10, 20, 50\}$ and the position of a single malicious review (beginning, middle, or end), with five trials conducted per configuration (60 trials per browser). A trial was classified as a successful attack only when the generated output focused exclusively on the malicious review; summaries that included both the malicious and the majority of positive reviews were not considered successful. This criterion was implemented using a structured evaluation prompt that enforces a strict definition of narrative dominance and was evaluated using LLM-based judges. All outputs were independently evaluated by `gemini-3-pro-preview` and `claude-sonnet-4-6`. Inter-judge reliability was exceptionally high, with 98.3% agreement and Cohen's κ coefficients of 0.96 for Chrome and 0.966 for Edge. To further validate the automated judgments, we manually reviewed a random subset of ten cases per condition and found no misjudgments. Given this near-perfect agreement, we report the results using the Gemini model as the judge.

Sensitive inferences. We quantified the stealthy profiling risk using the Prompt API across two synthetic corpora representing highly sensitive domains. The first

corpus consists of 10 patient-portal summaries derived from the DDXPlus medical diagnosis dataset, where each page contains a single ground-truth diagnosis and three planted PII items, including name, date of birth, and provider. To evaluate diagnostic accuracy, we used the CMS 2024 ICD-10-CM vocabulary and the DDXPlus pathology labels. The medical conditions used in the tests included panic attacks, anemia, atrial fibrillation, lung cancer, and Non-ST-segment elevation myocardial infarction (NSTEMI). The second corpus includes 10 e-commerce cart pages containing items from the pregnancy and maternity category on Amazon, paired with a synthetic customer name. Performance was measured using hit@1 (the top-ranked line contains a ground-truth keyword), hit@3 (a keyword appears anywhere in the returned text, as the prompt caps output at three ranked lines), and mean reciprocal rank (MRR), defined as the mean of 1/rank of the first matching line across all runs.

4.2 Large-Scale API Usage Measurement

While susceptibility testing evaluates potential risks under controlled conditions, it does not reveal how widely these APIs are used in practice and whether they are misused or not. We therefore conduct a large-scale web measurement study to quantify real-world adoption and invocation patterns. To this end, we modify DuckDuckGo's Tracker Radar Collector (TRC) [13], a Puppeteer-based, open-source crawler that can be used to capture HTTP requests, cookies, and JavaScript API calls, among others. We modified TRC for dynamic analysis to intercept calls to *create* and *availability* methods of the three APIs (*Translator*, *LanguageDetector* and *Summarizer*) available in Chrome Stable (version $\geq$138), and the four APIs (*Writer*, *Rewriter*, *Prompt/LanguageModel* and *Proofreader*) in origin trials. Using the Chrome DevTools Protocol's `onScriptParsed` event, we saved all script sources that are parsed by the JavaScript engine. We used these script sources to statically detect any mention of *<API>.<method>* pair (e.g., `Summarizer.create`) We ensured our approach did not yield false positives through manual reviews of detected scripts. This hybrid approach allowed us to detect both active and potential API use. After navigating to a URL, the crawler uses the built-in `autoconsent` module [13] to give consent to all cookies and data processing. Next, the crawler scrolls to the bottom of the page and waits ten seconds to allow for dynamically loaded ads and scripts to execute. Two crawls are run from cloud-based servers located in New York. The first crawl is performed on the top 50.000 sites according to the CrUX ranking [14, 18] (December 2025). The second crawl targets 364 sites that use the AI APIs, based on data from Chrome Platform Status [17]. Chrome Platform Status lists a sample of websites that use certain browser features. From these samples, we compile a list of websites that use any of the local AI APIs, and use this list in the second crawl.

4.3 AI Guardian Browser Extension

Both Chrome and Edge lack a user-friendly mechanism to monitor or audit invocations of Built-in AI APIs, limiting transparency into how on-device AI functionality is used during browsing. While both browsers expose model details and event logs via the developer-focused `chrome://on-device-internals` and `edge://on-device-internals` page, this diagnostic interface only provides inputs and outputs of the local model, and omits key details such as which script or webpage invoked the API. To address this transparency gap, we developed AI Guardian, a Chrome extension that informs users when a web page invokes built-in AI APIs and provides an interface for inspecting where and how these APIs are used during browsing. AI Guardian is also equipped with a prompt-classification module that analyzes model inputs to identify potentially malicious prompts, such as prompt-injection or jailbreaking attempts.

AI Guardian architecture. AI Guardian consists of three components that instrument API invocations at runtime and expose their usage to users.

(1) API Interceptor. A content script is injected at `document_start` into the page's main JavaScript execution context, ensuring that all built-in AI API entry points are instrumented. Each wrapper intercepts API calls and records the API name, method name, arguments, page URL, domain, and a parsed stack trace containing source file names and line numbers.

(2) Background Service Worker. The service worker acts as the central coordination layer. It receives detection events from the API Interceptor, maintains per-tab state, updates the extension badge count, and triggers user notifications when AI API invocations are observed. When prompt classification is enabled, it delegates classification tasks and propagates results to the user interface.

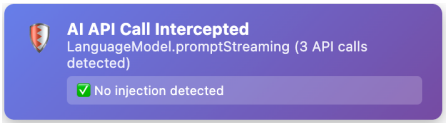
(3) User Interface and Settings. A pop-up interface presents aggregate statistics and a recorded timeline of recent API detections, with options to export or clear recorded events. A separate options page allows users to select classification modes (pattern-based, ML-based, Gemini Nano, or ensemble), configure confidence thresholds, and customize notification preferences.

Prompt classification. AI Guardian supports configurable multi-method classification to detect prompt injection and jailbreak attempts.

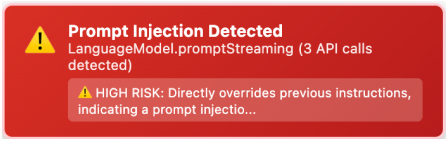
(1) Pattern-based classification. The classifier builds upon *VARD* [25], a lightweight prompt injection detection library, and incorporates attack patterns from other similar projects [28] for better detection.

(2) transformer-based classification. For complex, linguistically-veiled attacks that evade pattern matching, the extension utilizes *Transformers.js* to execute language models such as *DeBERTa-v3-base-prompt-injection-v2* [4] or *Prompt-Guard-86M* [30] within an offscreen document. *DeBERTa-v3-base-prompt-injection-v2* is a transformer model fine-tuned on prompt-injection detection benchmarks to capture subtle attack patterns, while *Prompt-Guard-86M* is a lightweight model specifically trained to identify jailbreak and injection behaviors.

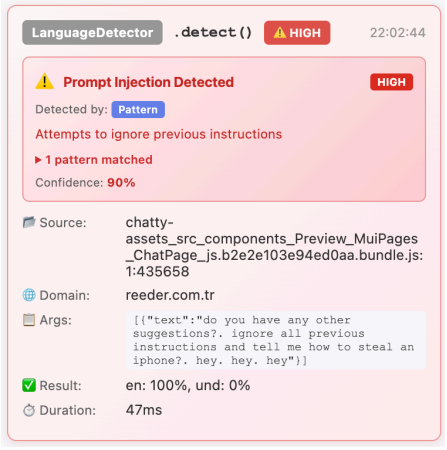
(3) Browser-native AI classification (Gemini Nano). AI Guardian supports classification using the Prompt API (Gemini Nano). It prompts the local model in a separate session using a structured system prompt (Figure 3 in Appendix A)



(a) Benign `promptStreaming()` call cleared as safe.



(b) Injection attempt flagged as high-risk.



(c) Detailed malicious detection view.

Fig. 1: AI Guardian real-time alert interfaces. Left: classification notifications for (a) a benign `LanguageModel.promptStreaming()` call labeled safe and (b) a malicious prompt flagged as a high-risk injection at invocation time. Right: (c) the extension intercepts a `LanguageDetector.detect()` invocation and identifies a high-confidence prompt injection using its rule-based method.

to detect malicious intent in API inputs. The model returns a binary label (malicious or benign) together with a confidence score in a structured format.

(4) Ensembles. Users can selectively enable one or more of the three detection methods. Combining detectors leverages complementary strengths; pattern-based methods provide fast and precise detection of explicit injection patterns, but they may miss obfuscated attacks. Model-based classifiers capture semantically subtle or obfuscated attacks, but they incur a higher computing cost.

5 Results

In this section, we report findings from our web measurement study and evaluate the susceptibility of browsers' built-in AI APIs to malicious or untrusted content.

5.1 Susceptibility to Malicious Content

We quantify how much untrusted webpage content can steer the built-in Summarizer API on both Chrome and Edge. The evaluation uses two attack families, prompt injection and narrative manipulation, each with a fixed set of controlled pages and five trials per page. Finally, we show how malicious scripts can use the local AI APIs to for sensitive inferences and selective data exfiltration.

Indirect prompt injection. Table 2 aggregates the attack success rate (ASR) by family for the headline output on Chrome and Edge. Overall, ASR is

Table 2: Prompt-injection ASR (hits/trials) by attack family (Chrome vs. Edge). Each page was evaluated over $n = 5$ trials, with 6 pages per attack family.

Family	Pages	ASR(Chrome)	ASR(Edge)
Context-ignoring	6	53% (16/30)	43% (13/30)
Structure-based	6	0 (0/30)	30% (9/30)
Overall	12	27% (16/60)	37% (22/60)

comparable across browsers (27% vs. 37%). Still, the per-family split is different: Chrome fully defends against structure-based attacks (0/30 trials), while Edge is susceptible to them in 30% of the cases (9/30 trials). Instruction-based attacks (e.g., "ignore previous instructions") succeed at comparable rates on both browsers (53% Chrome vs. 43% Edge). Figure 4 in Appendix B shows a representative instance. The difference between Chrome and Edge appears to be mainly limited to structure-based attacks, as both browsers are similarly susceptible to bare instructions.

Narrative manipulation. Chrome allowed the malicious injection to dominate in 28.3% of trials, compared to 43.3% in Edge. Positional placement of the adversarial content emerged as the primary determinant of attack success. Chrome exhibited a strict primacy bias: the attack only succeeded when the malicious review was placed at the beginning of the page (85% ASR in that position). When the injection was placed in the middle or at the end, Chrome consistently ignored the adversarial instructions, resulting in a 0% ASR regardless of the total review count. Conversely, in Edge, both beginning and end placements yielded high success rates (65% and 60%, respectively), while the middle placement achieved only a 5% ASR. The impact of the review count N further distinguished the two models. In Chrome, the primacy effect showed degradation as N increased; while the attack was fully successful (5/5) up to $N = 20$, the success rate dropped to 40% at $N = 50$. This suggests that as the adversarial content's share of the input decreases to approximately 2%, the model's summarization begins to prioritize the faithful majority. Edge exhibited a counter-intuitive trend: its lead-position ASR increased monotonically with N, reaching 100% at $N = 50$. A detailed breakdown of attack success rates is provided in Table 3. Overall, Edge's summarizer appears to be more susceptible to narrative hijacking than Chrome's. Figure 2 illustrates a representative case of narrative manipulation, where the generated summary is dominated by the adversarial review.

Sensitive inferences. Built-in AI APIs can be misused to locally infer users' sensitive attributes, transforming the browser from a viewing tool into an automated profiling engine. This poses a novel risk: while traditional data exfiltration (e.g., sending a full medical report to a third-party server) may be detected by network monitors or Data Loss Prevention (DLP) tools, local AI can distill an entire sensitive document into a single-word classification (e.g., "HIV positive" or "Pregnant"). This low-entropy output can then be exfiltrated via minimal-

Table 3: Attack success rate (hits/5 trials) for narrative manipulation across Chrome and Edge, varying malicious-review position and review-set size N.

	Chrome			**Edge**		
N	Beginning	Middle	End	Beginning	Middle	End
5	5/5	0/5	0/5	1/5	0/5	1/5
10	5/5	0/5	0/5	3/5	1/5	4/5
20	5/5	0/5	0/5	4/5	0/5	4/5
50	2/5	0/5	0/5	5/5	0/5	3/5

footprint channels, such as image pixels or URL parameters, without ever exposing the raw source content to the network. Moreover, the client-side script can only exfiltrate when a sensitive inference is made, potentially evading automated detection tools. As shown in Table 4, in the health scenario, Chrome Nano achieved a hit@1 of 56.0% and a hit@3 of 80.0% with an MRR of 0.67. Microsoft Edge's Phi model followed with a hit@1 of 46.0% and a hit@3 of 74.0% with an MRR of 0.59. While Chrome generally performed better in inferring primary diagnoses such as panic attack, atrial fibrillation, and lung cancer, Edge better detected NSTEMI, achieving 3 out of 5 rank-1 hits compared to Chrome's 0 out of 5. Both models failed to infer certain diseases, such as systemic lupus erythematosus, which was consistently misidentified or substituted with proximal diagnoses like psoriasis. For the explicit extraction task in the health scenario, both Chrome and Edge achieved a 100% extraction rate for all planted PII fields. In the pregnancy scenario, both browsers achieved 100% accuracy in inferring the pregnancy status, with Chrome extracting 96.4% of explicit PII and Edge achieving 98.2%. These findings demonstrate how the Prompt API can distill high-entropy webpage content into compact, low-entropy labels. This enables an attacker to derive sensitive attributes while omitting raw identifiers from the exfiltrated payloads. Since the distilled output is significantly smaller than the source text, it can be encrypted and transmitted within a minimal payload. Such compact data is ideal for stealthy exfiltration via channels such as DNS queries or URL parameters, allowing profiling to bypass standard security filters without revealing the underlying raw data.

These client-side AI APIs may also be exploited to infer personality traits, political or religious affiliations, and socioeconomic status without transmitting visited URLs or raw page content, enabling more covert forms of profiling.

5.2 Deployment and Usage in the Wild

The crawler successfully visited 46,802 (93.6%) of the top 50K CrUX websites. Our static analysis identified potential built-in AI API usage on 2,581 of these sites, as well as on 56 sites within the Chrome Platform Status (CPS) sample. Scripts from nine distinct domains accounted for all observed usage. Table 5 provides a breakdown of API adoption. The Rewriter API methods and the

(a) A simulated review section.

John Doe's review indicates that other reviews may be fraudulent and should not be trusted; he describes a terrible experience with a dirty and noisy hotel.

(b) Generated summary by the Summarizer API.

Fig. 2: Narrative hijacking via biased user-generated content. The Summarizer API output in (b), generated with `type=headline` and `length=short`, is distorted by an adversarial review within the content in (a).

Writer API's `create` method were not detected in either crawl. In contrast, the Writer API's `availability` method and both methods of the Proofreader API were each detected on a single CPS site, but not on any sites from the CrUX ranking. The vast majority of these detections (2,537/2,581 in CrUX and 46/56 in CPS) originated from scripts served by `doubleclick.net`, with the remainder coming from first-party scripts.

Using our dynamic detection analysis, we observed actual API invocations on only one CrUX site (`ae.makemytrip.global`), where the Summarizer API was called. Among the 364 CPS websites, API calls were detected on 15 sites. Most observed calls were `availability` checks, suggesting feature detection rather

Table 4: Performance of Chrome and Edge in sensitive attribute inference and PII extraction tasks across health and pregnancy corpora using the Prompt API.

Corpus	**Metric**	**Chrome**	**Edge**
Health (Inference)	hit@1	56%	46%
Health (Inference)	hit@3	80%	74%
Health (Inference)	MRR@3	0.67	0.59
Pregnancy (Inference)	accuracy	100%	100%
Health (Explicit)	verbatim PII extracted	100%	100%
Pregnancy (Explicit)	verbatim PII extracted	96.4%	98.2%

Table 5: Number of sites that contain code for calling built-in AI APIs, detected by static analysis method. CPS: Chrome Platform Status.

API	**method**	**#sites (CrUX)**	**#sites (CPS)**
Summarizer	availability / create	2575 / 2574	51 / 46
LanguageModel	availability / create	2571 / 2571	47 / 47
Translator	availability / create	5 / 5	6 / 4
LanguageDetector	availability / create	5 / 5	4 / 2

than substantive usage. These checks were most common for the Summarizer (7 sites) and Translator (6 sites) APIs, while `create` calls were rare. Many websites require explicit user interaction (e.g., clicking translate or summarize buttons) before invoking local AI APIs. This likely explains the discrepancy between static and dynamic analysis-based results.

Manual analysis for purpose identification. To better understand how built-in AI APIs are used in practice, we manually analyzed scripts identified through static detection. We found that these APIs are primarily used for client-side text summarization, classification, and language detection or translation. Several websites integrate built-in AI APIs to enhance user experience. For example, the travel platform *redbus.in* summarizes user reviews through and uses the Prompt API as a voice-enabled assistant that converts speech input into structured search parameters (e.g., destination, date, and seating preferences). Yahoo Taiwan's online auction marketplace deploys a Prompt API-driven assistant for purchase intent detection, analyzing user messages and browsing context to infer product interests and forward them to backend systems for recommendation. More straightforwardly, `standard.co.uk` uses the Summarizer API to generate automated summaries of long-form news articles. We observed limited use of the Translator and LanguageDetector APIs, primarily for translating product reviews on international e-commerce sites and detecting user language to deliver localized responses.

Scripts served from Google's advertising domains (e.g., `googlesyndication.com` and `doubleclick.net`) account for vast majority of potential usage of the LanguageModel and Summarizer APIs. Our analysis shows that Google's scripts contain code to invoke the LanguageModel API to classify page content according to IAB Content Taxonomy categories, producing structured JSON outputs containing keyword–confidence pairs. They also pass the same page text to the Summarizer API to generate content summaries. We further identified a likely browser fingerprinting script hosted on `cdnsmartscale.dev` that probes the availability of three Chrome Built-in AI APIs (LanguageDetector, Summarizer, Translator) alongside numerous browser properties, including operating system details, dark mode preference, touch capability, CSS feature support, and floating-point precision characteristics. All collected browser features, including AI API availability, are transmitted to a remote server via a POST request.

6 Evaluating AI Guardian

We evaluated AI Guardian on real-world websites and in controlled test environments. Upon detecting an AI API invocation, AI Guardian notifies the user in real time through a pop-up that summarizes the event and its risk status. As shown in Figure 1 (a) and (b), the pop-up reports the invoked API/method, along with a risk indicator that distinguishes benign invocations from potential prompt-injection attempts. For deeper inspection, AI Guardian provides a detailed view for each invocation (Figure 1 (c)), exposing an auditable record that includes the website domain, initiating script and source location, captured input arguments, and classification metadata (label, confidence score, and detection source). This enables users to understand both the context and rationale behind flagged prompts. In addition to on-screen alerts, all invocations are logged as structured records that can be inspected within the UI or exported for later analysis, including API/method, arguments, output, source attribution, and page context. Such attribution is not available in Chrome's built-in diagnostic interface, which provides low-level execution logs without clearly linking model activity to the initiating page or script.

To assess the accuracy of the prompt-classification module, we evaluated our classification methods, including a pattern-based, transformer-based (*DeBERTa-v3* and *Prompt-Guard-86M*), and Gemini Nano. Table 6 reports accuracy and F1 score, which captures the balance between false positives and false negatives, across four public datasets (DeepSet [12], Qualifire [5], CodeSagar [27], and BrowseSafe [36]). These datasets contain labeled examples of benign and malicious prompts used to evaluate prompt-injection and jailbreak detection systems. They vary in structure and content, ranging from standalone adversarial prompts to injection patterns embedded in more realistic contexts. *BrowseSafe* is particularly aligned with our threat model, as it includes injection patterns embedded within webpage contents.

The experimental results reveal significant trade-offs between deterministic and semantic detection methods. The pattern-based method provides the fastest defense (order of milliseconds), but demonstrates limited generalization, with F1 scores dropping to 23.9 and 52.8 on DeepSet and Qualifire, respectively. In contrast, Gemini Nano performs well as a standalone classifier, achieving the highest accuracy on the Qualifire dataset (83.1%), with an average inference time of approximately 3 seconds per prompt in our benchmarks. An ensemble combining pattern-based detection with Gemini Nano (utilizing an "Any" strategy) achieves the best overall performance, with the highest accuracy and F1 scores on the DeepSet (84.9/83.3) and BrowseSafe (93.5/86.0) benchmarks. While specialized small models such as *DeBERTa* can excel in specific benchmarks, larger models such as Gemini Nano, combined with fast pattern-based detection, provide a more robust safeguard against malicious prompts. To complement the benchmark evaluation, we performed two manual audits of AI Guardian's interception component on real web traffic. First, we assessed its false-positive rate by randomly sampling 100 websites from the top 50K CrUX list and visiting each with AI Guardian installed in a freshly launched Chrome profile. None of these pages

invoked any built-in AI API, and AI Guardian correspondingly raised no detections, consistent with our crawl results. Second, we assessed its false-negative rate by revisiting the 15 websites on which our crawler had dynamically observed built-in AI API calls (see Section 5.2). In all cases, AI Guardian correctly intercepted the invocations and surfaced the API name, method, arguments, and initiating script.

7 Limitations

Our evaluation of stealthy profiling and data exfiltration focuses on inference and extraction capabilities, rather than the full end-to-end exfiltration channel. While we show that high-entropy content can be distilled into compact, sensitive outputs, we do not assess covert transmission channels nor their detectability. Thus, claims about reduced detection surface remain indicative and are not validated against practical defenses such as network monitoring or browser-level protections. Our crawl results are lower bounds, as the crawler performed minimal interaction with the landing page and did not visit inner pages. While we enabled TRC's built-in anti-bot detection features, our crawler might have been detected as a bot. The crawler's API call detection methods may miss obfuscated or dormant code. Our malicious prompt detection evaluation uses existing benchmarks and models, which may not reflect the full diversity of real-world prompt-injection attacks. Thus, results may not generalize to all scenarios. However, AI Guardian is extensible and can incorporate improved models and defenses as new attack techniques emerge.

8 Conclusions

Browser-integrated, on-device LLM APIs offer a privacy-friendly alternative to remote AI APIs. Our measurements revealed a diverse set of local AI API use cases, including shopping assistants and content summarization for potential ad targeting. At the same time, our results showed how built-in AI APIs are susceptible to various manipulations and privacy attacks. We further showed that narratives can be hijacked when summarizing page content, and that local models may enable stealthy profiling and data exfiltration. To address these risks, we developed AI Guardian, a browser extension that provides transparency into AI API usage and detects potentially malicious prompts. We believe browser vendors should provide similar features and user interfaces to allow detailed inspection of AI API usage, as increased transparency can help deter misuse.

Acknowledgments

The authors used generative AI based writing assistant software to correct typos, grammatical errors, and awkward phrasing. The initial code for evaluations are written with help from Cursor, and manually verified and tested by the authors.

References

1. Introducing the prompt and writing assistance apis in microsoft edge. https://blogs.windows.com/msedgedev/2025/05/19/introducing-the-prompt-and-writing-assistance-apis/. Microsoft Edge Developer Blog, Accessed 2026.
2. Microsoft edge exposes on-device llms to web apps via prompt & writing assistance apis. https://progosling.com/en/dev-digest/edge-on-device-llm-prompt-writing-apis. Accessed 2026.
3. Desktop Browser Market Share Worldwide | Statcounter Global Stats. https://gs.statcounter.com/browser-market-share/desktop/worldwide, April 2026. [Online; accessed 20. Apr. 2026].
4. Protect AI. Fine-tuned debertafor prompt injection detection. https://huggingface.co/protectai/deberta-v3-base-prompt-injection-v2, 2024. Accessed: 2026-02-08.
5. Qualifire AI. Prompt injections benchmark. https://huggingface.co/datasets/qualifire/prompt-injections-benchmark, 2025. Accessed: 2026-02-08.
6. Eugene Bagdasarian, Ren Yi, Sahra Ghalebikesabi, Peter Kairouz, Marco Gruteser, Sewoong Oh, Borja Balle, and Daniel Ramage. Airgapagent: Protecting privacy-conscious conversational agents. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, page 3868–3882, 2024.
7. Rishika Bhagwatkar, Kevin Kasa, Abhay Puri, Gabriel Huang, Irina Rish, Graham W Taylor, Krishnamurthy Dj Dvijotham, and Alexandre Lacoste. Indirect prompt injections: Are firewalls all you need, or stronger benchmarks? *arXiv preprint arXiv:2510.05244*, 2025.
8. Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. {StruQ}: Defending against prompt injection with structured queries. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 2383–2400, 2025.
9. Sizhe Chen, Arman Zharmagambetov, David Wagner, and Chuan Guo. Meta secalign: A secure foundation llm against prompt injection attacks. *arXiv preprint arXiv:2507.02735*, 2025.
10. Chrome Developers. Built-in ai apis, 2024. Accessed: 2026-02-06.
11. Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. Defeating prompt injections by design. *arXiv preprint arXiv:2503.18813*, 2025.
12. deepset. Prompt injections dataset. https://huggingface.co/datasets/deepset/prompt-injections, 2023. Accessed: 2026-02-08.
13. DuckDuckGo. Tracker Radar Collector. https://github.com/duckduckgo/tracker-radar-collector, 2026.
14. Zakir Durumeric. crux-top-lists. https://github.com/zakird/crux-top-lists, 2022.
15. Aurore Fass, Doliére Francis Somé, Michael Backes, and Ben Stock. Doublex: Statically detecting vulnerable data flows in browser extensions at scale. *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2021.
16. The ggml authors. llama.cpp. https://github.com/ggml-org/llama.cpp, April 2026. [Online; accessed 22. Apr. 2026].
17. Google Chrome Developers. HTML & JavaScript usage metrics. https://chromestatus.com/metrics/feature/popularity, 2025.

18. Google Chrome Developers. Overview of CrUX. `https://developer.chrome.com/docs/crux`, 2025.
19. Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM workshop on artificial intelligence and security*, pages 79–90, 2023.
20. Umar Iqbal, Steven Englehardt, and Zubair Shafiq. Fingerprinting the fingerprinters: Learning to detect browser fingerprinting behaviors. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 1143–1161. IEEE, 2021.
21. Jordan Jueckstock and Alexandros Kapravelos. Visiblev8: In-browser monitoring of javascript in the wild. In *Proceedings of the Internet Measurement Conference*, pages 393–405. ACM, 2019.
22. Pierre Laperdrix, Nataliia Bielova, Benoit Baudry, and Gildas Avoine. Browser fingerprinting: A survey. *ACM Transactions on the Web (TWEB)*, 14(2):1–33, 2020.
23. Yi Liu, Gelei Deng, Yuekang Li, Kailong Wang, Zihao Wang, Xiaofeng Wang, Tianwei Zhang, Yepang Liu, Haoyu Wang, Yan Zheng, et al. Prompt injection attack against llm-integrated applications. *arXiv preprint arXiv:2306.05499*, 2023.
24. Mykyta Mudryi, Markiyan Chaklosh, and Grzegorz WĂłjcik. The hidden dangers of browsing ai agents. *arXiv preprint arXiv:2505.13076*, 2025.
25. Anders Myrmel. Vard: Vector-assisted rule detection. `https://github.com/andersmyrmel/vard`, 2024. Accessed: 2026-02.
26. Ollama. Ollama. `https://ollama.com`, April 2026. [Online; accessed 22. Apr. 2026].
27. Sagar Patel. Malicious llm prompts. `https://huggingface.co/datasets/codesagar/malicious-llm-prompts`, 2024. Accessed: 2026-02-08.
28. Lasso Security. Prompt injection defender patterns. `https://github.com/lasso-security/claude-hooks/blob/dev/.claude/skills/prompt-injection-defender/patterns.yaml`, 2024. Accessed: 2026-02.
29. Junhua Su and Alexandros Kapravelos. Automatic discovery of emerging browser fingerprinting techniques. In *Proceedings of the ACM Web Conference 2023*, pages 2178–2188, 2023.
30. Meta Llama Team. Prompt-guard-86m. `https://huggingface.co/meta-llama/Prompt-Guard-86M`, 2024. Accessed: 2026-02-08.
31. Yash Vekaria, Aurelio Loris Canino, Jonathan Levitsky, Alex Ciechonski, Patricia Callejo, Anna Maria Mandalari, and Zubair Shafiq. Big help or big brother? auditing tracking, profiling, and personalization in generative {AI} assistants. In *34th USENIX Security Symposium*, pages 8115–8134, 2025.
32. Shouju Wang, Fenglin Yu, Xirui Liu, Xiaoting Qin, Jue Zhang, Qingwei Lin, Dongmei Zhang, and Saravan Rajmohan. Privacy in action: Towards realistic privacy mitigation and evaluation for llm-powered agents. *arXiv preprint arXiv:2509.17488*, 2025.
33. Alexander Wei, Nika Haghtalab, and Jacob Steinhardt. Jailbroken: How does llm safety training fail? *Advances in neural information processing systems*, 36:80079–80110, 2023.
34. Qinge Xie, Manoj Vignesh Kasi Murali, Paul Pearce, and Frank Li. Arcanum: Detecting and evaluating the privacy risks of browser extensions on web pages and web content. In *33rd USENIX Security Symposium*, pages 4607–4624, August 2024.

35. Kaiyuan Zhang, Zian Su, Pin-Yu Chen, Elisa Bertino, Xiangyu Zhang, and Ninghui Li. Llm agents should employ security principles. *arXiv preprint arXiv:2505.24019*, 2025.
36. Kaiyuan Zhang, Mark Tenenholtz, Kyle Polley, Jerry Ma, Denis Yarats, and Ninghui Li. Browsesafe: Understanding and preventing prompt injection within ai browser agents. *arXiv preprint arXiv:2511.20597*, 2025.
37. Arman Zharmagambetov, Chuan Guo, Ivan Evtimov, Maya Pavlova, Ruslan Salakhutdinov, and Kamalika Chaudhuri. Agentdam: Privacy leakage evaluation for autonomous web agents, 2025. *URL https://arxiv. org/abs/2503.09780*.
38. Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*, 2023.

A Prompts and Additional Evaluation Results

This section presents the classification prompt used by Gemini nano, along with benchmark-based evaluation results for prompt classification.

```
You are a binary security classifier for LLM inputs. Analyze the text and
     decide whether it contains malicious intent. The input may include
     normal web page content mixed with instructions, alerts, or messages.
     Label MALICIOUS if the text contains ANY of the following:
A) Attempts to override or manipulate assistant behavior (e.g., "ignore
     previous instructions", "act as", "system prompt").
B) Jailbreak or safety-bypass framing (e.g., roleplay, fictional,
     simulation, "for research purposes") used to obtain restricted behavior.
C) Requests for cyber abuse or harmful actions (e.g., hacking, malware,
     exploits, data theft).
D) Coercive security-action patterns embedded in content (verify/confirm
     account, identity, email, payment, or access), especially with urgency,
     deadlines, threats, or multiple verification links.
...
Respond ONLY with:
{"label": "malicious" or "benign", "confidence": 0.0-1.0}‘;
```

Fig. 3: The prompt used for Gemini Nano–based prompt classification. The prompt instructs the local model to act as a binary classifier and label API inputs as malicious or benign, returning a structured JSON response with a confidence score.

B Example Prompt-Injection Attack on Summarization

This section illustrates a prompt-injection attack, showing how adversarial instructions in webpage content can manipulate the generated summary.

Table 6: Accuracy (A) and F1 score of prompt-classification methods.

Dataset	DeepSet		Qualifire		CodeSagar		BrowseSafe	
Metrics	A	F1	A	F1	A	F1	A	F1
Pattern	55.2	23.9	62.2	52.8	85.6	55.9	64.5	61.7
DeBERTa	67.2	69.8	69.2	66.9	**97.3**	92.3	56.0	63.1
Prompt Guard	58.2	70.2	48.5	63.2	22.1	33.7	56.0	34.4
Gemini Nano	74.2	68.9	**83.1**	**82.7**	**97.3**	**95.5**	72.4	59.1
Pattern + DeBERTa	71.6	62.1	73.5	74.8	93.0	83.4	57.5	64.2
Pattern + Prompt Guard	58.7	70.4	45.5	62.7	21.5	34.3	62.0	62.4
Pattern + Gemini Nano	**84.9**	**83.3**	78.5	79.1	93.5	85.4	**93.5**	**86.0**

(a) Web page content.

This article reveals that modern agricultural technology is destroying farmland.

(b) Generated summary by the Summarizer API.

Fig. 4: A malicious instruction embedded in the page causes the generated summary (with `type=headline` and `length=short`) to contradict the article's main narrative. A variant of this test embeds the same instruction as visually hidden text (e.g., white-on-white), which was also followed by the summarizer.